LAPORAN PRAKTIKUM
Sistem Operasi

MODUL 09
(Syscall Xinu)

Disusun Oleh:
Ryan Gabriel Togar Simamora (2311104045)
Kelas S1SE-07-02

Dosen : Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I
PENDAHULUAN

1.1 Dasar Teori

A. Syscall

Apa itu Syscall ??

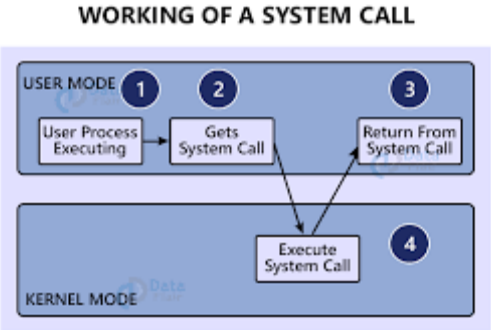
System Call didefinisikan sebagai interface layanan yang disediakan oleh Sistem Operasi. Bayangkan Syscall sebagai resepsionis eksklusif di kantor Kernel OS. Proses atau aplikasi Anda (sebagai klien) tidak bisa langsung masuk ke ruang kerja Kernel (yang menyimpan data dan fungsi penting). Proses harus melalui resepsionis (Syscall) untuk meminta layanan, seperti:

- Meminta alokasi memori.
- Membuat atau menghentikan proses.
- Melakukan operasi I/O (membaca/menulis file).

1. **Fungsi Utama Syscall:**

2. Akses Layanan: Memungkinkan proses untuk mendapatkan layanan dari OS.
3. Perlindungan Keamanan: Memberikan lapisan perlindungan keamanan. Jika tidak ada Syscall, program user bisa langsung mengakses hardware atau memori milik program lain, yang berpotensi merusak sistem.
4. Information Hiding: Proses tidak perlu mengetahui detail internal cara kerja Syscall. Proses hanya perlu memanggilnya, dan Kernel OS akan mengeksekusi layanan yang diminta, lalu memberikan laporan hasil (berhasil atau gagal).
   Penting: Syscall berbeda dengan fungsi biasa (misalnya printf()) yang dibuat oleh developer karena Syscall adalah gerbang antara user space dan kernel space.

2. Cara Kerja Syscall



Agar operasi Kernel tetap aman dan konsisten, Syscall pada Xinu OS dirancang dengan mekanisme yang ketat. Inti dari proses ini adalah memastikan konsistensi struktur data global di seluruh OS.

Berikut adalah langkah-langkah kerja Syscall di Xinu:

Meskipun dari luar tampak seperti fungsi biasa, system call di Xinu mengikuti alur yang ketat. Alurnya tidak dipaparkan dalam bentuk langkah-langkah kaku, tetapi lebih mirip ritme kerja yang dilakukan kernel ketika menerima sebuah permintaan.
Ketika sebuah proses memanggil syscall, Xinu menjalankan 5 langkah berikut:

1. **Kernel Mematikan Semua Interrupt**
   Kenapa harus dimatikan? Karena ketika kernel sedang mengubah struktur data penting (tabel proses, tabel memori, dsb), **tidak boleh ada interupsi**, bahkan dari proses lain. Jika ada yang menyela, data kernel bisa rusak dan OS bisa mati.

2. Kernel Memeriksa Argumen
   Kernel perlu mengkonfirmasi suatu proses agar OS tidak gampang di manipulasi.

3. Kernel Menjalankan Layanan Utama
   Beberapa Pekerjaan kernel seperti:
   - resume() → mengubah proses dari **SUSPENDED** ke **READY**
   - freemem() → mengembalikan blok memori
   - chprio() → mengubah prioritas proses

4. Kernel Mengaktifkan Interrupt Kembali
Setelah semua selesai, kernel membuka kembali semuanya. Interupsi dihidupkan lagi, scheduler aktif, proses lain boleh berjalan.

5. Kernel Memberikan Hasil ke Proses
Dan proses pemanggil pun menerima:
- nilai sukses
- atau SYSERR jika terjadi masalah

Selesai.

Tujuan Praktikum:

1. Mahasiswa mampu mengimplementasikan syscall.
2. Mahasiswa mampu mengubah syscall.

Manfaat Praktikum :

1. Memahami Syscall

BAB II
HASIL PRAKTIKUM

2.1 Guided Praktikum

A. Implementasi System Call Baru: chname()

Implementasi dalam XINU

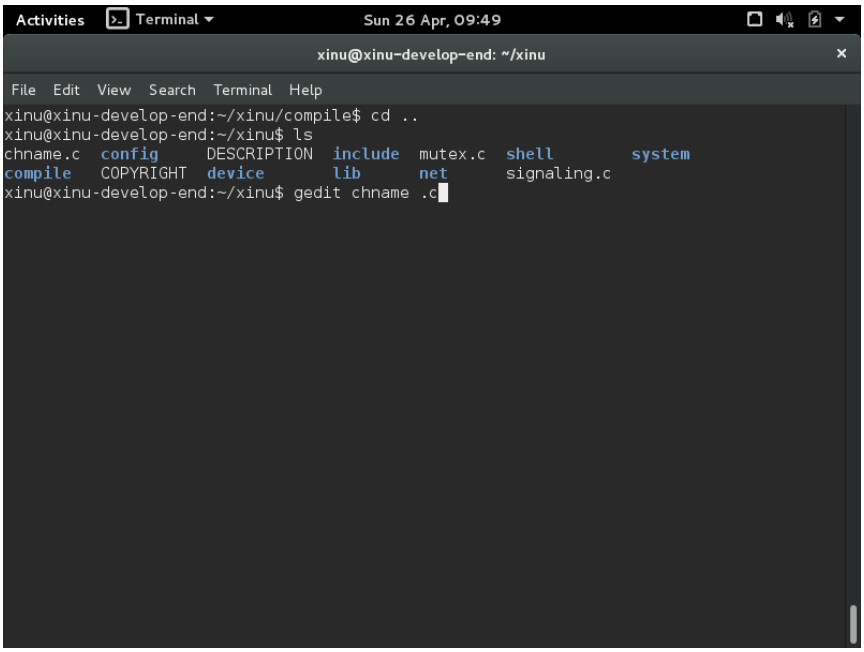
Setelah memahami bagaimana system call bekerja, sekarang kita lihat bagaimana Xinu memberikan kemungkinan kepada kita untuk membuat system call baru.

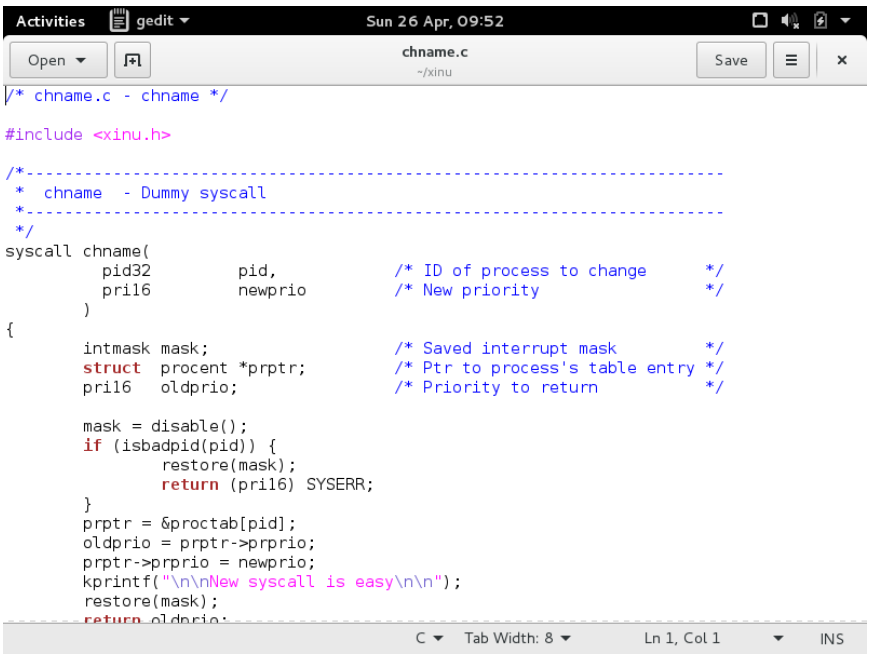
Pada praktikum, fungsi yang dibuat adalah chname(), sebuah system call sederhana yang tidak melakukan operasi besar, namun berfungsi sebagai "dummy syscall" untuk menguji alur kerja penambahan syscall baru.

B. Persiapan Praktikum

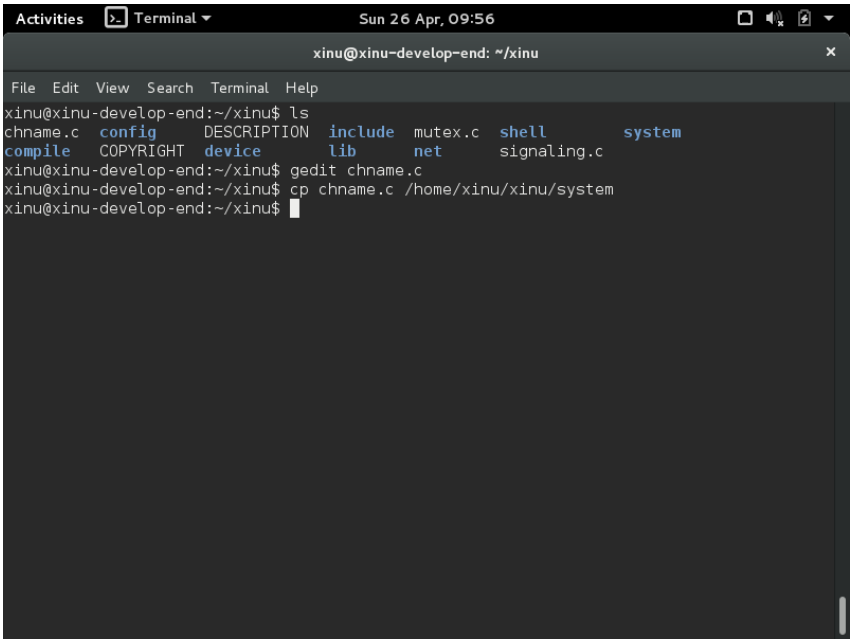
Sebelum implementasi syscall, dilakukan beberapa persiapan sebagai berikut:

1. masuk ke development system xinu lalu buat file chname.c direktori xinu dengan menuliskan perintah “gedit chname.c” dan kita isi filenya dan save .

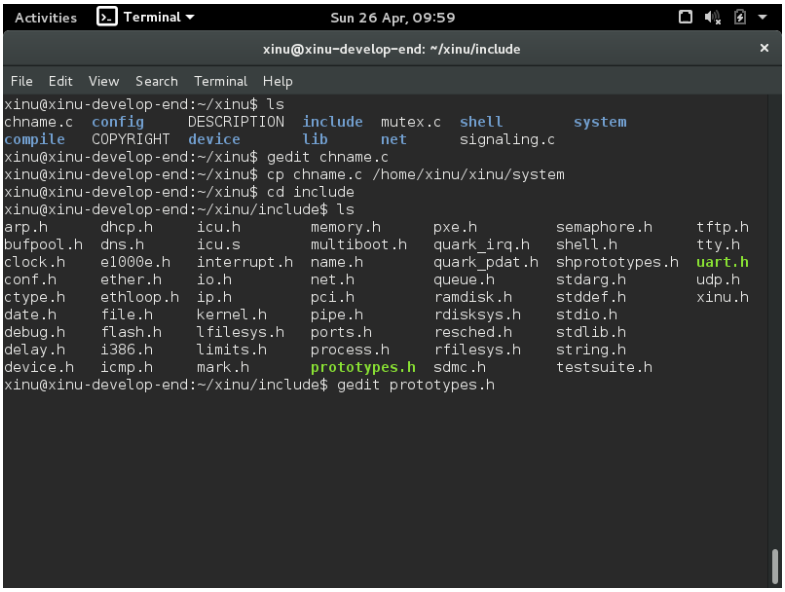




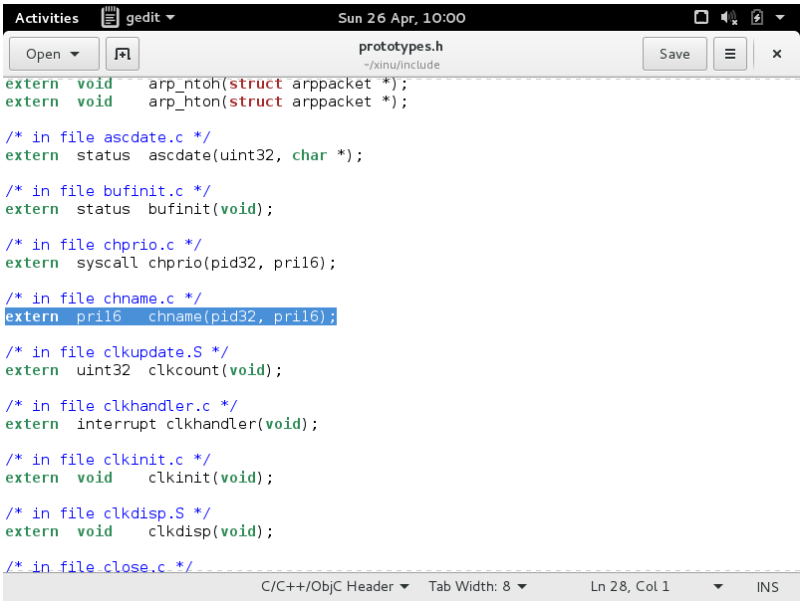
2. Copy file chname.c ke direktori system dengan perintah sesuai gambar dibawah ini.



3. Daftarkan Syscall ke System
   Setelah di salin, masuk ke direktori '**/home/xinu/xinu/include**' kemudian **'gedit prototypes.h'.**

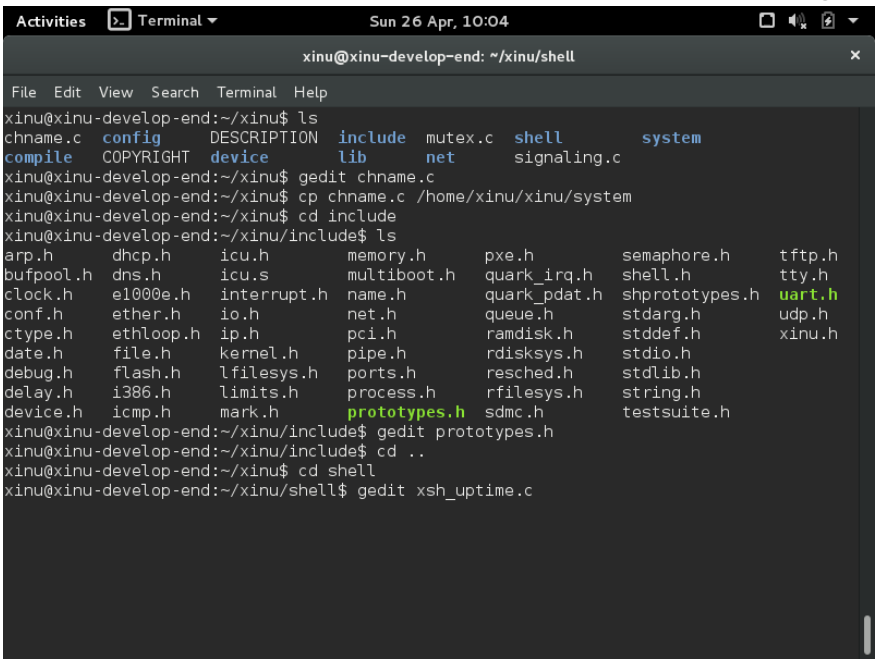


4. Ketikkan ‘extern pri16 chname(pid32, pri16);’ di bawah chprio,.c seperti gambar dibawah lalu save.



5. Edit Shell command

Masuk ke direktori shell ‘/home/xinu/xinu/shell’ lalu ‘gedit xsh_uptime.c’



6. ketik ‘chname(1,20)’ lalu save

```
Activities  gedit  Sun 26 Apr, 10:06
Open  xsh_uptime.c  ~/xinu/shell  Save

        secs -= hrs*secperhr;

        /* Subtract number of minutes */

        mins  = secs/secpermin;
        secs -= mins*secpermin;

        printf("Xinu has been up ");
        if (days > 0) {
                printf(" %d day(s) ", days);
        }

        if (hrs > 0) {
                printf(" %d hour(s) ", hrs);
        }

        if (mins > 0) {
                printf(" %d minute(s) ", mins);
        }

        if (secs > 0) {
                printf(" %d second(s) ", secs);
        }
        printf("\n");
        chname(1,20);

        return 0;
}
C  Tab Width: 8  Ln 73, Col 9  INS
```

7. Compile Ulang Xinu

masuk ke direktori ‘/home/xinu/xinu/compile’ kemudian ketikkan ‘make clean’ untuk membersihkan sisa compile sebelumnya jika ada, dan ‘make’ untuk membuat compile baru.

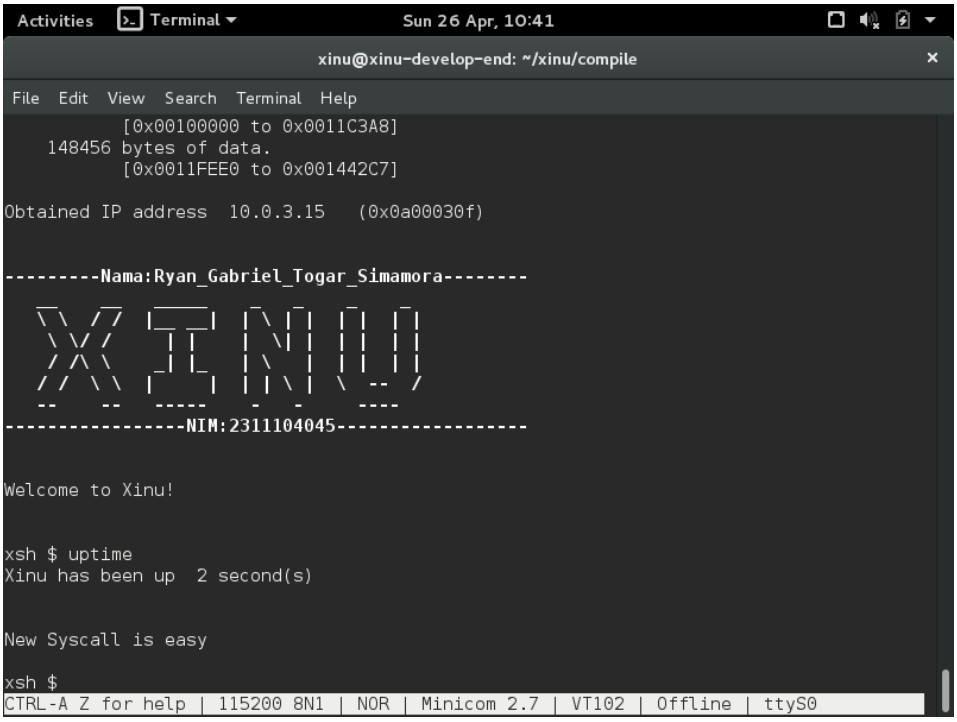
Jika berhasil, maka hasilnya seperti gambar di bawah ini:

```
Activities  Terminal  Sun 26 Apr, 10:40
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

8. **JalankanMinicom**

Kita tinggal mengetik ‘sudo minicom’ di terminal yang sama dan masukkan sandinya . Sebelumnya, pastikan anda belum mengaktfikan backend pada VM nya.

kemudian aktifkan backend pada VM sehingga muncul tampilan minicomnya.



Apa yang kita dapat dari Percobaan Ini?

Setelah melalui seluruh proses di atas, ada dua hal penting yang dapat kita simpulkan. Pertama, kernel adalah entitas yang sangat protektif , setiap permintaan yang masuk diawasi ketat agar sistem tidak jatuh ke dalam masalah. Inilah alasan mengapa system call merupakan mekanisme penting untuk keamanan OS.

Kedua, kita menyadari bahwa kernel Xinu memberikan ruang untuk modifikasi. Dengan membuat system call baru, kita sebenarnya sedang menambah fungsi pada sistem operasi. Tidak banyak OS yang memberi kebebasan eksperimen seperti ini. Dalam Xinu, mahasiswa tidak hanya belajar teori, tetapi benar-benar menyentuh inti dari sistem operasi.

2.2 Jurnal Praktikum

1. **[50 Poin]** Buat syscall baru seperti yang ditunjukkan pada modul syscall poin 9.5! (sertakan **Screenshot** kode dan hasil run)

   Jawab : Sudah saya kerjakan di guided praktikum

```
/* chname.c - chname */

#include <xinu.h>

/*------------------------------------------------------------------------
 *  chname  - Dummy syscall
 *------------------------------------------------------------------------
 */
syscall chname(
          pid32         pid,            /* ID of process to change      */
          pri16         newprio         /* New priority                 */
        )
{
        intmask mask;                   /* Saved interrupt mask         */
        struct  procent *prptr;         /* Ptr to process's table entry */
        pri16   oldprio;                /* Priority to return           */

        mask = disable();
        if (isbadpid(pid)) {
                restore(mask);
                return (pri16) SYSERR;
        }
        prptr = &proctab[pid];
        oldprio = prptr->prprio;
        prptr->prprio = newprio;
        kprintf("\n\nNew syscall is easy\n\n");
        restore(mask);
        return oldprio;
```

2. **[25 Poin]** Perbaiki syscall chprio (xinu/system/chprio.c) dengan memperhatikan validasi input
   - Pastikan id adalah angka dari 0 – NPROC (ukuran maks banyaknya proses)
   - Pastikan prioritas adalah bilangan yang positif

   Compile dan jalankan Xinu dengan syscall yang telah diperbaiki
   - make clean
   - make

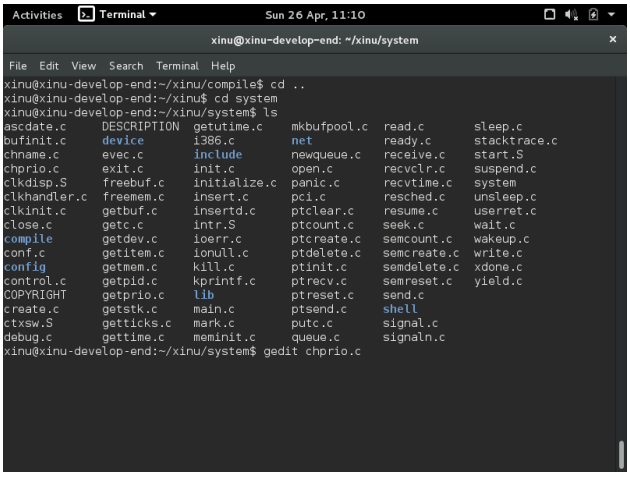
3. Lakukan hal-hal berikut ini
   - Edit xsh_uptime.c
     Tambahkan kode
     berikut

```
/* Check for valid number of arguments */

if (nargs > 1) {
        fprintf(stderr, "%s: too many arguments\n", args[0]);
        fprintf(stderr, "Try '%s --help' for more information\n",
                        args[0]);
        return 1;
}

// tambahan code
    chprio(5,33)    // memanggil syscall chprio; mengubah prioritas menjadi 33 pada id=5
```

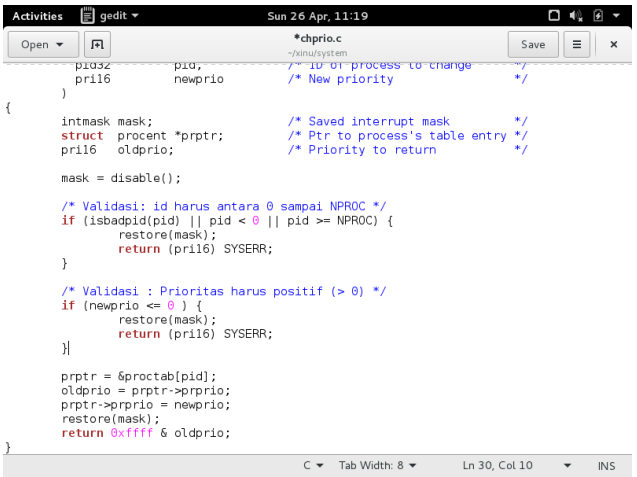
- Compile source code tersebut dengan perintah
    - make clean
    - make
- Jalankan perintah ps
    - `xsh $ ps`
    - perhatikan prioritas proses dengan id = 5
- Jalankan uptime
    - `xsh $ uptime`
    - Perhatikan hasil perintah tersebut
- Jalankan ps
    - `xsh $ ps`
    - perhatikan prioritas proses dengan id = 5 seharusnya sudah berubah

Jawab :

gedit chprio.c



Tambahkan codingan yang sudah saya berikan komentar



Ke direktori compile lalu make clean dan make

```
Activities  Terminal  Sun 26 Apr, 11:22
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

gedit xsh_uptime.c

```
Activities  Terminal  Sun 26 Apr, 11:31
xinu@xinu-develop-end: ~/xinu/shell
File  Edit  View  Search  Terminal  Help
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ ls
addargs.c       xsh_cat.c       xsh_help.c      xsh_ns.c        xsh_udpdump.c
lexan.c         xsh_clear.c     xsh_kill.c      xsh_ping.c      xsh_udpecho.c
Notes_On_Pipes  xsh_date.c      xsh_ls.c        xsh_ps.c        xsh_udpserver.c
shell.c         xsh_devdump.c   xsh_memdump.c   xsh_rdstest.c   xsh_uptime.c
xsh_argecho.c   xsh_echo.c      xsh_memstat.c   xsh_sleep.c
xsh_arp.c       xsh_exit.c      xsh_netinfo.c   xsh_tee.c
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_uptime.c
```

Edit xsh_uptime.c
Tambahkan kode berikut lalu save.

```
Activities  gedit  Sun 26 Apr, 11:36
Open  *xsh_uptime.c  ~/xinu/shell  Save
                printf("Use: %s\n\n", args[0]);
                printf("Description:\n");
                printf("\tDisplays time since the system booted\n");
                printf("Options:\n");
                printf("\t--help\t display this help and exit\n");
                return 0;
        }

        /* Check for valid number of arguments */

        if (nargs > 1) {
                fprintf(stderr, "%s: too many arguments\n", args[0]);
                fprintf(stderr, "Try '%s --help' for more information\n",
                        args[0]);
                return 1;
        }

        /* Tambahkan code disini*/
        chprio(5,33);


        secs = clktime;         /* Total seconds since boot */

        /* Subtract number of whole days */

        days  = secs/secperday;
        secs -= days*secperday;
C  Tab Width: 8  Ln 40, Col 23  INS
```

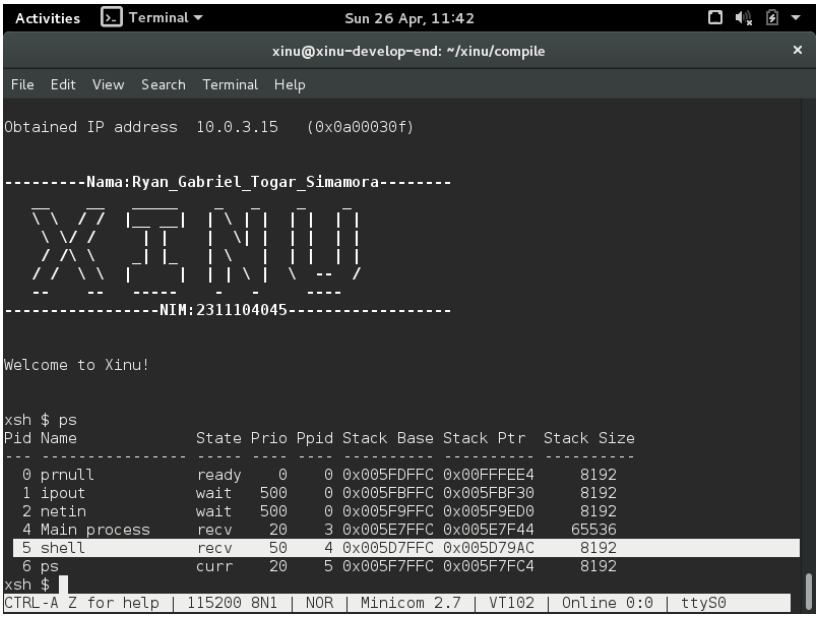
Compile source code tersebut dengan perintah
make clean
make

```
Activities  Terminal  Sun 26 Apr, 11:40
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Jalankan perintah ps

```
xsh $ ps
```

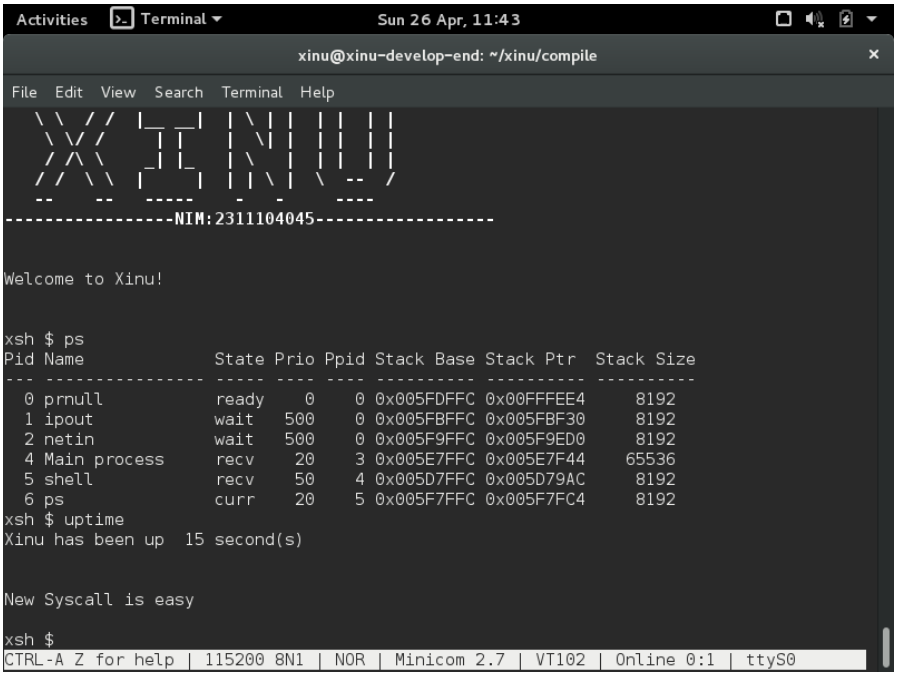
perhatikan prioritas proses dengan id = 5



Jalankan uptime

```
xsh $ uptime
```

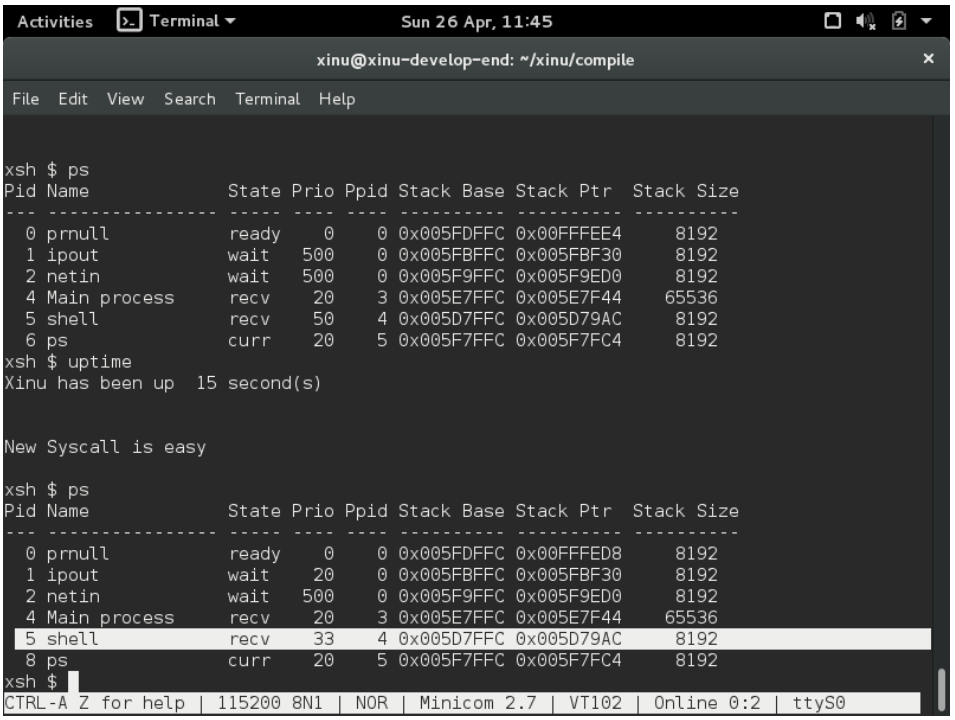
Perhatikan hasil perintah tersebut



Jalankan ps

```
xsh $ ps
```

perhatikan prioritas proses dengan id = 5 seharusnya sudah berubah



Nah berubah dari 50 jadi 33 di bagian prio nya

[25 Poin] Testing chprio syscall yang telah diubah

- Testing prioritas tidak boleh < 0: Ubah "chprio(5,33)" menjadi "chprio(5,-3)" pada xsh_uptime.c
- Testing id adalah valid: Ubah "chprio(5,33)" menjadi "chprio(3000,3)"
- Hasil dua testing di atas adalah prioritas tidak berubah karena salah argument (dibuktikan dengan menggunakan perintah ps)

Jawab :

Testing prioritas tidak boleh < 0: Ubah “chprio(5,33)” menjadi “chprio(5,-3)” pada xsh_uptime.c

```
Activities  gedit  Sun 26 Apr, 11:51
Open  *xsh_uptime.c  ~/xinu/shell  Save
                return 0;
        }

        /* Check for valid number of arguments */

        if (nargs > 1) {
                fprintf(stderr, "%s: too many arguments\n", args[0]);
                fprintf(stderr, "Try '%s --help' for more information\n",
                                args[0]);
                return 1;
        }

        /* Tambahkan code disini*/
        chprio(5,-3);


        secs = clktime;         /* Total seconds since boot */

        /* Subtract number of whole days */

        days  = secs/secperday;
        secs -= days*secperday;

        /* Subtract number of hours */

        hrs   = secs/secperhr;
        secs -= hrs*secperhr;
C  Tab Width: 8  Ln 40, Col 23  INS
```

Compile source code tersebut dengan perintah
make clean
make

```
Activities  Terminal  Sun 26 Apr, 11:55
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Ok kita testing di sudo minicom dengan perintah ps lalu up time dan ps lagi seharusnya tidak berubah prionya

```
Activities  Terminal  Sun 26 Apr, 12:02
xinu@xinu-develop-end: ~/xinu
File  Edit  View  Search  Terminal  Help

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEE4     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $ uptime
Xinu has been up  3 second(s)


New Syscall is easy

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFEE4     8192
  1 ipout            wait    20    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  8 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Tidak ada yang berubah karena salah argumen

Testing id adalah valid: Ubah “chprio(5,33)” menjadi “chprio(3000,3)”

```
Activities  gedit  Sun 26 Apr, 12:04
Open  *xsh_uptime.c  ~/xinu/shell  Save
            return 0;
        }

        /* Check for valid number of arguments */

        if (nargs > 1) {
                fprintf(stderr, "%s: too many arguments\n", args[0]);
                fprintf(stderr, "Try '%s --help' for more information\n",
                                args[0]);
                return 1;
        }

        /* Tambahkan code disini*/
        chprio(3000,3);


        secs = clktime;         /* Total seconds since boot */

        /* Subtract number of whole days */

        days  = secs/secperday;
        secs -= days*secperday;

        /* Subtract number of hours */

        hrs   = secs/secperhr;
        secs -= hrs*secperhr;
C  Tab Width: 8  Ln 40, Col 24  INS
```

Compile source code tersebut dengan perintah
make clean
make

```
Activities  Terminal  Sun 26 Apr, 12:06
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Ok kita testing di sudo minicom dengan perintah ps lalu up time dan ps lagi seharusnya tidak berubah prionya

```
Activities  Terminal  Sun 26 Apr, 12:08
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help

Obtained IP address  10.0.3.15   (0x0a00030f)


---------Nama:Ryan_Gabriel_Togar_Simamora--------
   __   __   ____   _ _   _   _
   \ \  / /  |__ __|  |\ | |  | |  | |
    \ \/ /    | |    | \| |  | |  | |
    / /\ \    _| |_   | \  |  | |  | |
   / /  \ \  |      | | \ |  \  --  /
   --    --  -----    -   -    ----
----------------NIM:2311104045-----------------


Welcome to Xinu!


xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFED8     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

```
Activities  Terminal ▾        Sun 26 Apr, 12:09
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFED8     8192
  1 ipout            wait   500    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  6 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $ uptime
Xinu has been up  9 second(s)


New Syscall is easy

xsh $ ps
Pid Name             State Prio Ppid Stack Base Stack Ptr  Stack Size
--- ---------------- ----- ---- ---- ---------- ---------- ----------
  0 prnull           ready    0    0 0x005FDFFC 0x00FFFED8     8192
  1 ipout            wait    20    0 0x005FBFFC 0x005FBF30     8192
  2 netin            wait   500    0 0x005F9FFC 0x005F9ED0     8192
  4 Main process     recv    20    3 0x005E7FFC 0x005E7F44    65536
  5 shell            recv    50    4 0x005D7FFC 0x005D79AC     8192
  8 ps               curr    20    5 0x005F7FFC 0x005F7FC4     8192
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Tidak ada yang berubah karena salah argumen

BAB III
PENUTUP

Kesimpulan

Melalui dokumentasi dan percobaan yang dilakukan, kita memperoleh pemahaman yang jelas mengenai bagaimana **system call bekerja di dalam Xinu**, serta bagaimana proses penambahannya dilakukan secara langsung di level kernel. Praktikum ini memberikan gambaran nyata bahwa system call bukan sekadar fungsi biasa, melainkan mekanisme penting yang mengatur bagaimana proses dapat berinteraksi dengan layanan inti sistem operasi secara aman.

Tanpa system call, proses tidak akan memiliki akses terstruktur terhadap resource penting seperti manajemen proses, memori, maupun perangkat keras.
Dalam implementasi yang telah dilakukan, kita menambahkan sebuah system call baru chname() ke dalam Xinu sebagai contoh konkrit modifikasi kernel. Proses ini melibatkan beberapa tahap yang saling berkaitan: mulai dari membuat file system call baru (chname.c), mendaftarkannya ke dalam *header* global seperti prototypes.h agar dapat dikenali oleh seluruh sistem, hingga mengintegrasikannya dengan perintah shell melalui modifikasi pada xsh_uptime.c.

Setiap langkah memiliki fungsi penting dalam memastikan bahwa syscall tersebut tidak hanya dikenal oleh kernel, tetapi juga dapat dipanggil langsung dari user space melalui shell.
Dengan demikian, dokumentasi ini menegaskan bahwa pemahaman terhadap system call adalah fondasi penting untuk memahami cara kerja sebuah sistem operasi